



Terraform Modules v3 User Guide

EC2 and VPC — using the modules and upgrading from v2 to v3

Prepared By:	Netcov
Prepared For:	Think Stack Infrastructure
Date:	August 26, 2026
Classification:	Internal

Table of Contents

1. Before You Start
2. How These Modules Are Used
3. The EC2 Modules
4. The VPC Module
5. What v3 Changes
6. Upgrading a Repo to v3
7. Reading the Plan



1. Before You Start

This guide covers two families of modules in the `thinkstack-co/terraform-modules` repository: the EC2 modules (`ec2_instance`, `ec2_domain_controller`, and the vendor appliances) and the `vpc` module. It explains what each module builds, how to call it, what changed in v3, and the exact steps to move a customer repository from v2 to v3.

You do not need prior knowledge of this repository. You do need to be comfortable reading a Terraform plan.

The four rules that govern this work

- **Customer repositories consume modules by Git ref.** Every module block pins a version with `?ref=vX.Y.Z`. Changing that ref is what an upgrade is.
- **Terraform Cloud runs every plan and apply.** You push to GitHub, TFC picks up the change, you review the plan in the TFC UI, and you approve. Do not run `terraform apply` from a laptop.
- `terraform plan` **is the safety gate**. Nothing in this guide is dangerous if you read the plan before approving it. Section 7 shows you what a good plan looks like.
- **One repository at a time.** These modules are shared by more than sixty repositories. Upgrade, verify, then move on.

2. How These Modules Are Used

A customer repository does not define AWS resources directly. It calls shared modules and passes in the customer-specific values. A module call looks like this:

Anatomy of a module call

```
module "app_server" {
  source = "github.com/thinkstack-co/terraform-modules//modules/aws/ec2_instance?ref=v3.0.0"

  name           = "prod-app-01"
  ami            = "ami-0abcdef1234567890"
  instance_type  = "t3.large"
  subnet_id      = module.vpc.private_subnet_ids[0]
  key_name       = aws_key_pair.prod.key_name

  vpc_security_group_ids = [module.app_server_sg.id]

  tags = {
    terraform    = "true"
    created_by   = "Terraform"
    environment  = "prod"
  }
}
```

Reading the source line

Here is a real one, from a live customer repo. When you upgrade a module, the version at the end is the only part of this line you touch — everything to its left stays exactly as it is.

[A module source line in a live customer repo](#)

```
source = "github.com/thinkstack-co/terraform-modules//modules/aws/ec2_instance?ref=v2.10.0"
```

▲ change only this

Part	Meaning
github.com/thinkstack-co/terraform-modules	The shared module repository.
//modules/aws/ec2_instance	The subdirectory holding the module. The double slash separates the repository from the path inside it.
?ref=v3.0.0	The Git ref to check out. This is the version pin.

Tags versus branches. ?ref=v3.0.0 points at a tag — an immutable snapshot. It will produce the same code forever. A ref such as ?ref=dev-backups points at a branch tip, which moves under you: the infrastructure definition changes every time someone commits to that branch. A handful of repositories are still pinned that way. Always pin production repositories to tags.

Where the module source lives

The modules are checked out locally at ~/Documents/GitHub/terraform-modules/modules/aws/. When a module call does not behave the way you expect, read the module's own variables.tf and main.tf rather than guessing from the call site. Every module also ships a README with examples.

The upgrade workflow, end to end



That is the whole loop. The rest of this guide is about knowing which ref to move to, and how to tell a safe plan from a dangerous one.

3. The EC2 Modules

3.1 ec2_instance — a single server

This module creates one EC2 instance and its monitoring. Specifically:

- One aws_instance with an encrypted root volume
- A CloudWatch alarm on the instance status check
- A CloudWatch alarm on the system status check

Both alarms can be turned off together with create_cloudwatch_alarms = false.

One module block equals exactly one instance. If a customer needs three application servers, that is three module blocks with three names. This is the single most important thing to know about this module — see Section 5.1 for why.

Minimal working example

```
module "file_server" {
  source = "github.com/thinkstack-co/terraform-modules//modules/aws/ec2_instance?ref=v3.0.0"

  name           = "prod-fs-01"
  ami            = data.aws_ami.windows_2022.id
  instance_type  = "m5.xlarge"
  subnet_id      = module.vpc.private_subnet_ids[0]
  key_name       = aws_key_pair.prod.key_name

  root_volume_size = 200
  root_volume_type = "gp3"
  http_tokens      = "required"

  vpc_security_group_ids = [module.file_server_sg.id]

  tags = local.tags
}
```

Options you will actually use

Variable	What it does	Default
name	Name tag on the instance and its root volume. Also used in alarm names.	required
ami	AMI ID. Changes are ignored after creation so a new AMI release does not replace a live server.	required
instance_type	Instance size.	t3.medium
subnet_id	Subnet to launch into.	required
vpc_security_group_ids	List of security group IDs.	[]
root_volume_size	Root volume size in GB.	100
root_volume_type	Root volume type. gp3 unless there is a reason.	gp3
encrypted	Encrypt the root volume.	true
http_tokens	IMDSv2 enforcement. See the warning below.	optional
disable_api_termination	Termination protection.	false
create_cloudwatch_alarms	Create the two status check alarms.	true
exclude_root_volume_snaps hot	Strip backup selection tags from the root volume so tag-based backup jobs do not pick it up separately.	false
tags	Tag map merged into everything the module creates.	see module

Set `http_tokens` explicitly. In v3 this variable defaults to `optional`, which allows both IMDSv1 and IMDSv2. In v2.9.2 it defaulted to `required`. A repository that relied on the default will quietly loosen instance metadata security when it upgrades. Put `http_tokens = "required"` in the module block when you upgrade unless something on the instance genuinely needs IMDSv1.

What it gives back

The useful outputs are `id`, `private_ip`, `private_dns`, `subnet_id`, `availability_zone`, and `primary_network_interface_id`. All are scalars — one instance, one value. Reference them as `module.file_server.private_ip`.

3.2 ec2_domain_controller — Windows domain controllers

This module builds the customer's domain controllers as a set, and wires the VPC's DHCP options to point at them. It creates:

- One EC2 instance per domain controller, named `<name>1`, `<name>2`, and so on
- A DHCP options set listing those DCs as the domain name servers, plus optional NTP servers
- A DHCP options association onto the VPC
- Instance and system status check alarms for each DC

Unlike `ec2_instance`, this module keeps the `number` variable — building a matched set of DCs is the whole point of it.

Two domain controllers across two AZs

```
module "domain_controllers" {
  source = "github.com/thinkstack-co/terraform-modules//modules/aws/ec2_domain_controller?ref=v3.0.0"

  name          = "prod-dc"
  number        = 2
  ami           = data.aws_ami.windows_2022.id
  instance_type = "t3.large"

  # Positional: index 0 is dc1, index 1 is dc2.
  private_ip = ["10.11.1.10", "10.11.2.10"]
  subnet_id  = [module.vpc.private_subnet_ids[0], module.vpc.private_subnet_ids[1]]

  domain_name = "customer.local"
  vpc_id      = module.vpc.vpc_id

  vpc_security_group_ids = [module.domain_controller_sg.id]

  tags = local.tags
}
```

The lists are positional. `private_ip[0]` and `subnet_id[0]` both belong to dc1. Keep them aligned, and keep them the same length as `number`. Getting this wrong puts a DC in the wrong subnet with the wrong address.

Turning off one DC without disturbing the other

v3 adds `disabled_dcs`. Listing a DC identifier there skips creating it, and — critically — leaves the other DCs' addresses in Terraform state untouched. Before v3, lowering number shifted every DC after the removed one.

```
number      = 2
disabled_dcs = ["dc2"] # builds dc1 only; dc1 is not disturbed
```

What it gives back

Outputs are **lists**, ordered dc1, dc2, and so on: `ec2_instance_id`, `ec2_instance_priv_ip`, `ec2_instance_subnet_id`, plus `dhcp_options_id`. Index them, for example `module.domain_controllers.ec2_instance_priv_ip[0]`.

3.3 Vendor appliance modules

Under `modules/aws/vendor/` are four modules that build network appliances from vendor AMIs. They are EC2 instances underneath, but each has vendor-specific network interface wiring, so they are not interchangeable with the general module.

Module	What it builds	v3 status
<code>fortigate_firewall</code>	Fortigate firewall with WAN, private, and DMZ interfaces. Feeds the VPC module's firewall routes.	Migrated in v3 — see Section 5.4
<code>cato_sdwan</code>	Cato SD-WAN socket.	Not yet migrated
<code>corelight</code>	Corelight network sensor.	Not yet migrated
<code>silverpeak</code>	Silverpeak SD-WAN appliance.	Not yet migrated; also has a known defect

The three unmigrated appliances still use the deprecated `inline_network_interface` block, which the AWS provider will remove in its v7 release. They are a separate piece of work from the EC2 and VPC upgrade described here, and they carry a higher risk because the change can force an instance replacement.

4. The VPC Module

This is the largest module in the repository and the foundation of every customer environment. One module block builds the entire network layer.

What it creates

- The VPC itself, an internet gateway, and DNS settings
- Up to six subnet tiers, each spread across your availability zones: `public`, `private`, `dmz`, `db`, `mgmt`, `workspaces`
- A route table per tier per AZ, with associations and default routes
- NAT gateways and Elastic IPs, or firewall routes if a firewall handles egress
- An S3 gateway endpoint associated to every private route table

- VPC flow logs to CloudWatch, with a KMS key, log group, IAM role and policy
- Optionally, SSM interface endpoints
- In v3: adoption and lockdown of the VPC's default security group

A standard three-AZ VPC

```
module "vpc" {
  source = "github.com/thinkstack-co/terraform-modules//modules/aws/vpc?ref=v3.0.0"

  name      = "customer_prod_vpc"
  vpc_cidr  = "10.11.0.0/16"
  azs       = ["us-east-1a", "us-east-1b", "us-east-1c"]

  private_subnets_list = ["10.11.1.0/24", "10.11.2.0/24", "10.11.3.0/24"]
  public_subnets_list  = ["10.11.201.0/24", "10.11.202.0/24", "10.11.203.0/24"]
  db_subnets_list      = ["10.11.11.0/24", "10.11.12.0/24", "10.11.13.0/24"]
  dmz_subnets_list     = ["10.11.101.0/24", "10.11.102.0/24", "10.11.103.0/24"]
  mgmt_subnets_list    = ["10.11.61.0/24", "10.11.62.0/24", "10.11.63.0/24"]
  workspaces_subnets_list = ["10.11.21.0/24", "10.11.22.0/24", "10.11.23.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = false

  tags = local.tags
}
```

Everything is positional against azs. The first entry in azs takes the first CIDR from every *_subnets_list, and the first entry in fw_network_interface_id. These lists must stay aligned and the same length. If you disable an AZ, you still leave its CIDR slot in place — removing it shifts everything after it onto the wrong addresses.

Egress: NAT gateways or a firewall

A VPC routes outbound traffic one of two ways, and the module supports both. You pick one.

Mode	Set	Result
NAT gateway	enable_nat_gateway = true	A NAT gateway and Elastic IP per AZ. Add single_nat_gateway = true to build only one and route all AZs through it — cheaper, but a single point of failure.
Firewall	enable_firewall = true	Default routes point at the firewall's network interfaces instead. Supply fw_network_interface_id and, for the DMZ tier, fw_dmz_network_interface_id.

In firewall mode the NIC lists are positional against azs, but a customer often has one firewall serving several AZs. In v3 the module wraps the list automatically, so a single NIC entry serves every AZ. See Section 5.3.

Leaving pieces out

Not every customer needs all six tiers in all AZs. There are three levels of control, and they combine.

Variable	Scope	Example
disabled_azs	Removes an AZ entirely — every tier, route table, NAT gateway and association for it.	disabled_azs = ["us-east-1c"]
<tier>_subnet_disabled	Removes one tier across every AZ, along with its route tables and routes.	dmz_subnet_disabled = true
<tier>_subnet_disabled_azs	Removes one tier in specific AZs only. Use when a tier does not need full spread.	db_subnet_disabled_azs = ["us-east-1c"]

An AZ is skipped for a tier if it appears in either `disabled_azs` or that tier's own disabled list. The CIDR slots for disabled AZs stay in the `subnets_list` variables regardless.

A worked v3 example: firewall egress across three AZs

This is the shape most customer environments take — a firewall handling outbound traffic instead of NAT gateways. Written the way v3 wants it.

```
module "vpc" {
  source = "github.com/thinkstack-co/terraform-modules//modules/aws/vpc?ref=v3.0.0"

  name      = "customer_prod_vpc"
  vpc_cidr  = "10.11.0.0/16"
  azs       = ["us-east-1a", "us-east-1b", "us-east-1c"]

  private_subnets_list = ["10.11.1.0/24", "10.11.2.0/24", "10.11.3.0/24"]
  public_subnets_list  = ["10.11.201.0/24", "10.11.202.0/24", "10.11.203.0/24"]
  db_subnets_list      = ["10.11.11.0/24", "10.11.12.0/24", "10.11.13.0/24"]
  dmz_subnets_list     = ["10.11.101.0/24", "10.11.102.0/24", "10.11.103.0/24"]
  mgmt_subnets_list    = ["10.11.61.0/24", "10.11.62.0/24", "10.11.63.0/24"]
  workspaces_subnets_list = ["10.11.21.0/24", "10.11.22.0/24", "10.11.23.0/24"]

  # The firewall carries egress, so no NAT gateways or EIPs are built.
  enable_nat_gateway = false
  enable_firewall    = true

  # One firewall serves all three AZs. v3 wraps this list, so a single entry
  # is enough — earlier versions needed the NIC repeated once per AZ.
  fw_network_interface_id = [module.firewall.private_network_interface_id[0]]
  fw_dmz_network_interface_id = [module.firewall.dmz_network_interface_id[0]]

  # Defaults to true. Set false only if the pre-flight check reported REVIEW.
  manage_default_security_group = true

  tags = local.tags
}
```

Two lines in that block are new in v3 and worth pointing at. The firewall interface lists carry a **single** entry rather than one per AZ — v3 wraps the list, so a lone firewall serves all three. And `manage_default_security_group` is stated explicitly rather than left to chance, because its default locks the VPC's default security group down to zero rules.

Older repos often repeat the same NIC once per AZ to work around the earlier behaviour. That still works under v3 and does not need cleaning up during an upgrade — but write new blocks the short way.

What it gives back

The outputs you will use most are `vpc_id`, `private_subnet_ids`, `public_subnet_ids`, and the other per-tier ID lists. There are also `*_by_az` map outputs keyed by availability zone name, for when position is not good enough:

```
subnet_id = module.vpc.private_subnet_ids[0]           # by position
subnet_id = module.vpc.private_subnets_by_az["us-east-1b"] # by AZ name
```

The `_by_az` outputs remain keyed by real AZ name in v3, even though the underlying state addresses changed. That was a deliberate part of the v3 design — see Section 5.3.

5. What v3 Changes

v3 is a cleanup release. Its themes are: stop using Terraform's count where it causes damage, ship the state migrations inside the modules so upgrades are hands-off, remove resources and arguments the AWS provider has deprecated, and tighten a few defaults.

Why count matters. With count, Terraform stores resources by position: `[0]`, `[1]`, `[2]`. Remove the middle one and everything after it shifts down a slot — Terraform then sees the wrong resource in each slot and destroys and recreates live servers to make reality match. Keying by a stable identifier instead removes that entire class of accident.

5.1 ec2_instance

Change	What it means for you
Only one instance per module block	The count on the instance is gone. <code>number</code> is still accepted when it is 1 or absent, and rejected with a clear error above that.
Ships its own state migration	A moved block inside the module renames the old <code>[0]</code> state address to the plain one. Upgrading from any older version costs zero destroys and needs no work in the customer repo.
<code>http_tokens</code> default changed	Was required, now optional. Set it explicitly when you upgrade.
Recover action removed from the system alarm	The alarm no longer attaches an EC2 recover action. EC2 simplified automatic recovery is on by default for supported instance types, and alerting runs through Datto.
<code>enable_recover_action</code> removed	So is the instance-type data source that fed it. Nothing in the fleet passes this variable.
Deprecated variables accepted quietly	<code>monitoring</code> , <code>root_iops</code> , <code>auto_recovery</code> , <code>region</code> and <code>number</code> are still accepted so existing module blocks upgrade unedited. The first two take precedence over their replacements; the rest are ignored.
Provider ceiling added	Now <code>>= 4.0.0</code> , <code>< 7.0.0</code> . This holds the fleet below AWS provider v7 until the S3 and CloudWatch deprecations elsewhere in the repo are dealt with.
Terraform floor raised	<code>>= 1.1.0</code> , which is what moved blocks require.

5.2 ec2_domain_controller

Change	What it means for you
DCs keyed by name, not position	State addresses moved from [0], [1] to ["dc1"], ["dc2"]. Turning one DC off no longer shifts the others.
Ships its own state migration	moved blocks covering up to six DCs are built into the module. The rename happens on the first plan after the upgrade, with no destroys.
disabled_dcs added	Skip specific DCs by identifier, for example ["dc2"], without disturbing the rest.
Outputs unchanged in shape	They are still lists in dc1, dc2 order, so existing references keep working.
Recover action removed from the system alarm	Same reasoning as ec2_instance.
DHCP name server order	Now built by iterating the keyed map, which sorts dc1 before dc2. Same result as before.

5.3 vpc

The VPC module received the largest change in v3, and the most carefully designed one.

Change	What it means for you
Network resources re-keyed	Subnets, route tables, routes, associations, NAT gateways and EIPs moved from positional keys to stable string keys derived from each AZ's original position.
198 moved blocks ship with the module	Covering all 33 resource families across up to six AZs. A repository on v2.9.2 or older migrates with zero destroys and no file added to the customer repo.
AZ-facing behaviour preserved	Name tags, the disable-by-AZ variables, and every *_by_az output still use real AZ names. Only the internal state address changed.
Default security group locked down	New manage_default_security_group, defaulting to true. The module adopts the VPC's existing default security group and strips it to zero rules. Section 6 covers the pre-flight check.
Firewall route defect fixed	When a customer supplies fewer firewall interfaces than AZs, the module now wraps the list instead of failing. Previously this errored — a single firewall serving three AZs could not plan.
Pre-flight script added	modules/aws/vpc/v3-preflight.sh — a read-only check to run before the upgrade. Section 6 covers it.

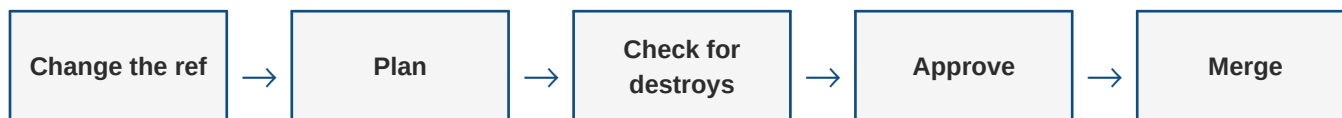
5.4 vendor/fortigate_firewall

The firewall's primary interface moved from the deprecated inline network_interface block to primary_network_interface. Two consequences:

- The module now requires AWS provider `>= 6.10.0`. Every repository with a firewall is pinned to that floor or higher.
- A `prevent_destroy` guard was added to the instance. If a plan would replace the firewall, Terraform errors out instead of proceeding. That is intentional and fail-safe: the firewall is never at risk from an accidental approval. If you hit that error, the change needs a maintenance window, not a workaround.

6. Upgrading a Repo to v3

The same five steps for every repository, whatever version it is coming from. The plan in step three is the safety gate — everything before it is reversible, and nothing after it happens without your approval.



Step 1 — Point the module at v3

Change the version at the end of every module source line. Nothing else in the module block needs to change: the deprecated variables are still accepted, so existing blocks upgrade unedited.

```

# Find every module ref in the repo
grep -rn "terraform-modules//modules/aws" --include="*.tf" .

# Before
source = "github.com/thinkstack-co/terraform-modules//modules/aws/ec2_instance?ref=v2.10.0"

# After
source = "github.com/thinkstack-co/terraform-modules//modules/aws/ec2_instance?ref=v3.0.0"
  
```

While you are in there, add `http_tokens = "required"` to any EC2 block that does not already set it — see Section 3.

Step 2 — Push, and let Terraform Cloud plan

Open a pull request. Terraform Cloud runs a speculative plan against it automatically. Nothing has changed in AWS at this point, and you can repeat this step as often as you like.

Step 3 — Confirm the plan destroys nothing

A correct v3 upgrade renames resources in state and leaves the infrastructure alone.

Do not approve anything that is going to destroy infrastructure.

Step 4 — Get approval

Have someone else read the plan and approve the pull request. A second pair of eyes on the destroy count is the point of this step.

Step 5 — Merge and apply

Merge the pull request. Terraform Cloud queues the real run against the workspace. Review it once more — it should match the plan you already read — then approve it. Confirm the environment is healthy before moving on to the next repository.

Two things to check first, if the repo has a VPC

Both apply only to the `vpc` module. EC2-only repositories can follow the five steps as written.

- **Run the pre-flight script.** `modules/aws/vpc/v3-preflight.sh` is read-only and takes a minute in CloudShell. It reports whether anything is still using the VPC's default security group, which v3 locks down. A REVIEW verdict means you should set `manage_default_security_group = false` in the module block before upgrading.
- **A VPC already on v2.10.0 needs a one-time state move.** Its network is stored under `availability-zone-name` keys, which v3 does not use, and the module's built-in migration cannot help it. Bumping the ref without moving state first produces a plan that destroys and recreates the entire network layer — which step three will catch. Move the state with `terraform state mv` first, generating the commands from `terraform state list` rather than typing them.



7. Reading the Plan

This is the step that keeps the upgrade safe. Everything below describes what you should see in Terraform Cloud after changing a ref to v3.

What a good upgrade looks like

The plan opens with a block of move statements, then reports no changes, or only in-place updates. Nothing is destroyed.

An `ec2_instance` upgrade — the ideal result

Terraform will perform the following actions:

```
# module.file_server.aws_instance.ec2 has moved to module.file_server.aws_instance.ec2
resource "aws_instance" "ec2" {
  id = "i-0abc123def4567890"
  # (30 unchanged attributes hidden)
}
```

Plan: 0 to add, 0 to change, 0 to destroy.



```
> ~ aws module.aws_prod_app004.aws_cloudwatch_metric_alarm.system[0]
> ~ aws module.aws_prod_app004.aws_instance.ec2[0] has moved to module.aws_prod_app004.aws_instance.ec2
> ~ aws module.aws_prod_appxtd.aws_cloudwatch_metric_alarm.instance[0]
> ~ aws module.aws_prod_appxtd.aws_cloudwatch_metric_alarm.system[0]
> ~ aws module.aws_prod_appxtd.aws_instance.ec2[0] has moved to module.aws_prod_appxtd.aws_instance.ec2
> ~ aws module.aws_prod_axway.aws_cloudwatch_metric_alarm.instance has moved to module.aws_prod_axway.aws_cloudwatch_metric_alarm.instance[0]
> ~ aws module.aws_prod_axway.aws_cloudwatch_metric_alarm.system has moved to module.aws_prod_axway.aws_cloudwatch_metric_alarm.system[0]
> ~ aws module.aws_prod_axway.aws_instance.ec2
> ~ aws module.aws_prod_blueb.aws_cloudwatch_metric_alarm.system[0]
> ~ aws module.aws_prod_blueb.aws_instance.ec2
> ~ aws module.aws_prod_bluepoint.aws_cloudwatch_metric_alarm.instance[0]
> ~ aws module.aws_prod_bluepoint.aws_cloudwatch_metric_alarm.system[0]
> ~ aws module.aws_prod_bluepoint.aws_instance.ec2[0] has moved to module.aws_prod_bluepoint.aws_instance.ec2
> ~ aws module.aws_prod_ca3.aws_cloudwatch_metric_alarm.system[0]
```

Terraform Cloud plan for an EC2 module upgrade showing the moved resources

```

> + aws module.vpc.aws_default_security_group.default[0]
> aws module.vpc.aws_route_table_association.db[0] has moved to module.vpc.aws_route_table_association.db["0"]
> aws module.vpc.aws_route_table_association.db[1] has moved to module.vpc.aws_route_table_association.db["1"]
> aws module.vpc.aws_route_table_association.dmz[0] has moved to module.vpc.aws_route_table_association.dmz["0"]
> aws module.vpc.aws_route_table_association.dmz[1] has moved to module.vpc.aws_route_table_association.dmz["1"]
> aws module.vpc.aws_route_table_association.private[0] has moved to module.vpc.aws_route_table_association.private["0"]
> aws module.vpc.aws_route_table_association.private[1] has moved to module.vpc.aws_route_table_association.private["1"]
> aws module.vpc.aws_route_table_association.public[0] has moved to module.vpc.aws_route_table_association.public["0"]
> aws module.vpc.aws_route_table_association.public[1] has moved to module.vpc.aws_route_table_association.public["1"]
> aws module.vpc.aws_route_table_association.workspaces[0] has moved to module.vpc.aws_route_table_association.workspaces["0"]
> aws module.vpc.aws_route_table_association.workspaces[1] has moved to module.vpc.aws_route_table_association.workspaces["1"]
> aws module.vpc.aws_route_table.db_route_table[0] has moved to module.vpc.aws_route_table.db_route_table["0"]
> aws module.vpc.aws_route_table.db_route_table[1] has moved to module.vpc.aws_route_table.db_route_table["1"]
> aws module.vpc.aws_route_table.dmz_route_table[0] has moved to module.vpc.aws_route_table.dmz_route_table["0"]

```

Terraform Cloud plan for a VPC upgrade showing the shipped moved blocks

Expected, and safe

What you see	Why	Action
has moved to lines, then 0 to destroy	The shipped moved blocks doing their job.	Approve.
In-place update to metadata_options	The http_tokens default changed. If you set it explicitly this will not appear.	Confirm the value is the one you want, then approve.
In-place update to a CloudWatch alarm's period	Alarm period tracks whether detailed monitoring is on.	Approve.
A system alarm losing its alarm_actions	The EC2 recover action was removed in v3, by design.	Approve.
No changes at all	The repo was already effectively on v3, or the moves are no-ops.	Approve.

Stop. Do not approve.

What you see	What it means
Any aws_instance being destroyed or replaced	A live server is about to be rebuilt. Nothing in this upgrade should do that. Stop and investigate.
Subnets, route tables or NAT gateways being destroyed	Almost certainly a VPC already on v2.10.0 that has not had its state moved yet. Discard the run and see the end of Section 6.

What you see	What it means
A security group appearing as a new resource	This is the default security group lockdown. It is an adoption, not a creation — but confirm the pre-flight said SAFE before letting it through.
An error mentioning <code>prevent_destroy</code>	A firewall would be replaced. Correct behaviour, blocking a real problem. Needs a maintenance window.
Anything you cannot explain	Ask. A plan you do not understand is not a plan you can approve.

